

UNIT :2

Concurrent Processes: Process Concept, Principle of Concurrency, Producer / Consumer Problem, Mutual Exclusion, Critical Section Problem, Dekker's solution, Peterson's solution, Semaphores, Test and Set operation; Classical Problem in Concurrency- Dining Philosopher Problem, Sleeping Barber Problem; Inter Process Communication models and Schemes, Process generation.

1 Process Concept

1.1 Definition

A **process** is a program that is currently executing.

When a program is stored on disk → it is passive.

When it runs in memory → it becomes a process.

Example:

`train_model.py` stored in folder → program

Running it using `python train_model.py` → process

1.2 Components of a Process

1. **Text Section** – Program code
2. **Data Section** – Global variables
3. **Stack** – Function calls, local variables
4. **Heap** – Dynamic memory
5. **PCB (Process Control Block)** – Stores process information

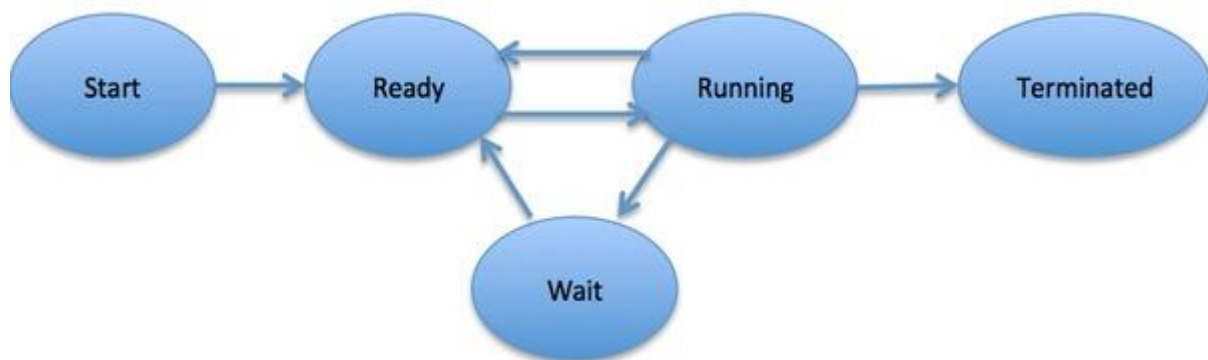
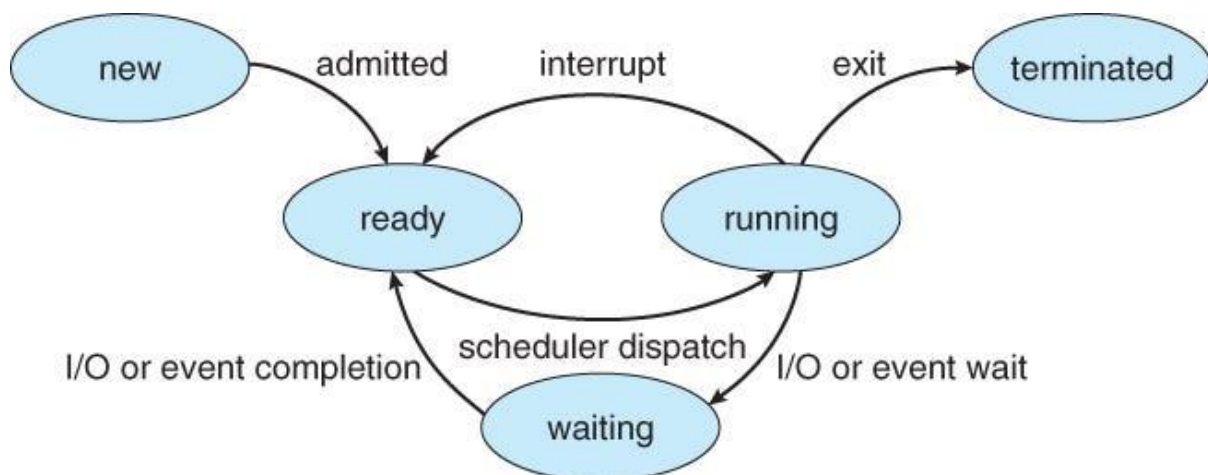
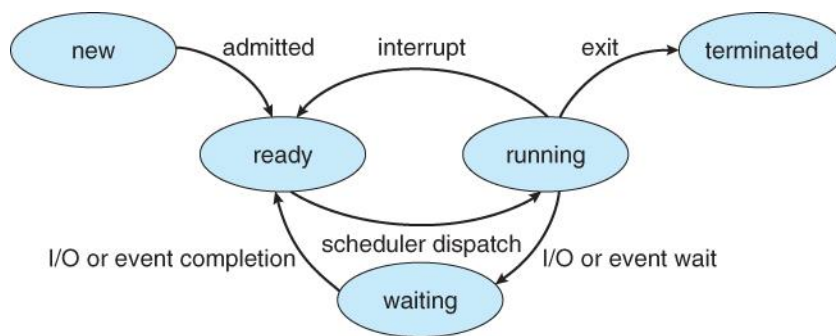
1.3 Process Control Block (PCB)

PCB contains:

- Process ID (PID)
- Process state
- Program counter
- CPU registers
- Memory info
- I/O status

PCB is like a **student record file** in college office.

1.4 Process States



States:

1. **New** – Process is being created
2. **Ready** – Waiting for CPU
3. **Running** – Currently executing
4. **Waiting (Blocked)** – Waiting for I/O
5. **Terminated** – Execution finished

Example:

While training ML model:

- Waiting for dataset → Waiting
- CPU doing calculations → Running

② Principle of Concurrency

Definition

Concurrency means multiple processes are making progress during the same time period.

It does NOT mean they run exactly at the same second.

Concurrency vs Parallelism

Concurrency	Parallelism
Logical overlapping	Physical simultaneous execution
Can happen on single CPU	Needs multiple cores
Uses context switching	True simultaneous execution

Common Mistake: Students treat both as same.

③ Critical Section Problem

3.1 Definition

A **critical section** is the part of a program where shared resources are accessed.

Example:

Two processes updating same variable:

```
count = count + 1
```

This is unsafe without synchronization.

3.2 Structure of Critical Section

```
Entry Section
Critical Section
Exit Section
Remainder Section
```

3.3 Conditions for Correct Solution

1. **Mutual Exclusion**
2. **Progress**
3. **Bounded Waiting**

These three are very important for exams.

4 Mutual Exclusion

Definition

Only one process can enter critical section at a time.

Example:

Only one person can use ATM machine at a time.

5 Producer–Consumer Problem

5.1 Definition

One process produces data.

Another process consumes data.

They share a common buffer.

5.2 Problem

- Producer should not produce if buffer is full.
- Consumer should not consume if buffer is empty.

5.3 Solution using Semaphore

We use three semaphores:

- mutex = 1
- empty = N
- full = 0

Producer:

```
wait(empty)
wait(mutex)
add item
signal(mutex)
signal(full)
```

Consumer:

```
wait(full)
wait(mutex)
remove item
signal(mutex)
signal(empty)
```

Real Engineering Example:

Data ingestion service → Producer

ML model → Consumer

Queue → Buffer

6 Semaphores

Definition

A semaphore is an integer variable used to control access to shared resources.

Types

1. Binary Semaphore (0 or 1)
2. Counting Semaphore (0 to N)

Operations

- wait() → decrement
- signal() → increment

If value < 0 → process waits.

Example:

Parking lot with 5 slots → counting semaphore = 5

7 Test and Set Operation

Definition

A hardware instruction that checks and sets lock in one atomic step.

Used in spinlocks.

Problem:

It causes **busy waiting** (CPU waste).

8 Dekker's Solution

Features:

- First software solution for mutual exclusion
- Works for only 2 processes
- Uses:
 - flag array

- turn variable

Limitation:

Not scalable.

9 Peterson's Solution

Improved version of Dekker.

Works only for 2 processes.

Uses:

- flag[2]
- turn variable

Logic:

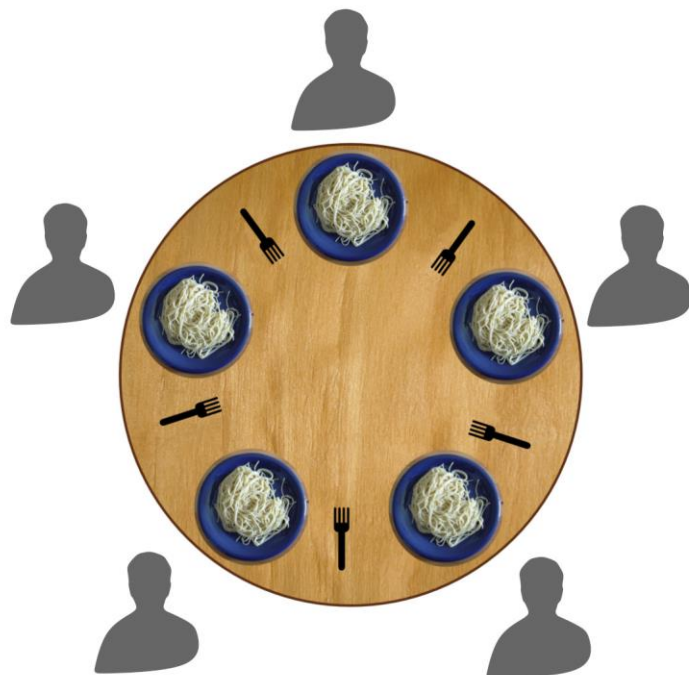
If both want to enter, turn decides who enters.

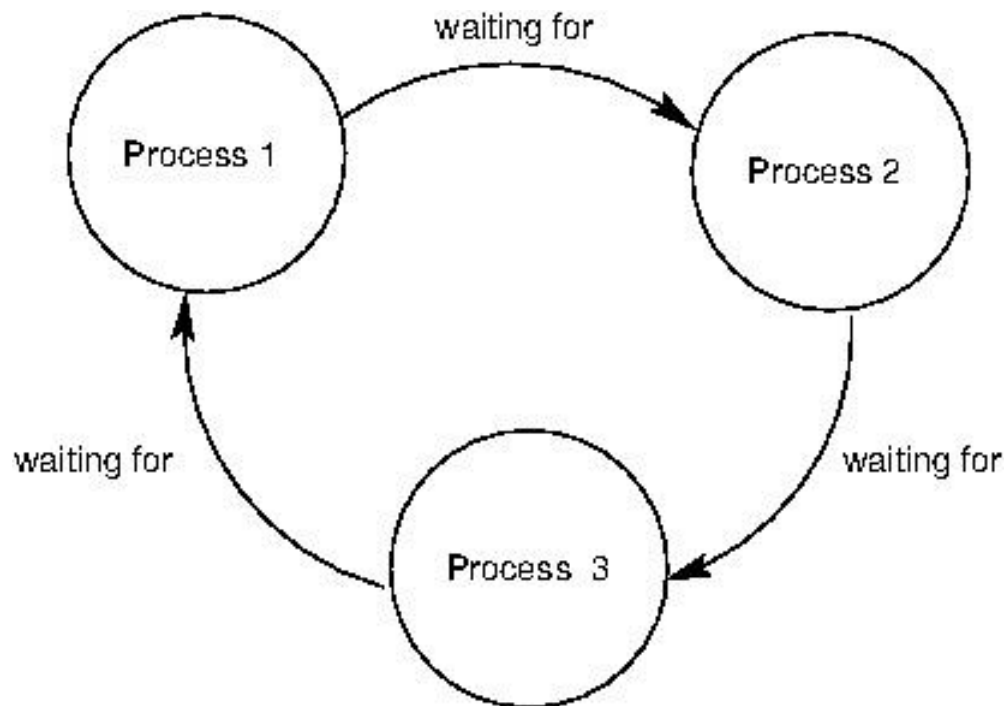
Limitation:

Only 2 processes supported.

10 Classical Problems in Concurrency

10.1 Dining Philosopher Problem





Problem:

5 philosophers

5 forks

Each needs 2 forks to eat

If all pick left fork → deadlock.

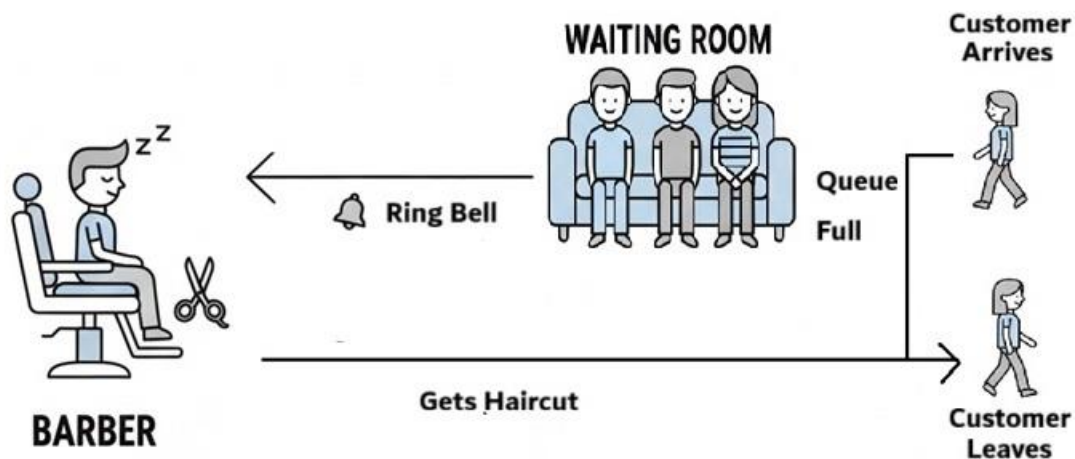
Concepts:

- Deadlock
- Mutual Exclusion
- Starvation

Solution:

- Allow only 4 philosophers at a time
- Use semaphores
- Change fork picking order

10.2 Sleeping Barber Problem



Problem:

- One barber
- Limited chairs
- If no customer → barber sleeps
- If chairs full → customer leaves

Concepts:

- Synchronization
- Resource management

1 1 Inter Process Communication (IPC)

Definition

IPC allows processes to exchange information.

Models of IPC

1. Shared Memory

- Processes share common memory.
- Fast communication.
- Needs synchronization.

Example:

Two C programs sharing array.

2. Message Passing

- Send() and Receive() functions.
- No shared memory.

- Safer.

Example:

Pipes, message queues, sockets.

Comparison

Shared Memory	Message Passing
Faster	Slower
Complex sync required	Simple
Same system	Can work across network

1 2 Process Generation

A parent process creates a child process.

System Calls:

- fork()
- exec()
- wait()

Example:

When you run a command in terminal,
Shell creates a child process.

Here are **important difference tables** from Concurrent Processes unit. These are exam-oriented and frequently asked.

1 Concurrency vs Parallelism

Basis	Concurrency	Parallelism
Meaning	Multiple tasks making progress in same time period	Multiple tasks executing at exact same time
CPU Requirement	Can work on single CPU	Requires multiple CPUs/cores
Execution	Context switching	True simultaneous execution

Goal	Manage multiple tasks efficiently	Increase speed using multiple cores
Example	Switching between Chrome & VS Code on 1 core	Training model on multi-core CPU

Common Mistake: Writing both as same.

2 Process vs Program

Basis	Program	Process
Definition	Passive set of instructions	Program in execution
Nature	Static	Dynamic
Location	Stored on disk	Loaded in memory
Resources	No resources allocated	CPU, memory allocated
Example	train.py file	Running train.py

3 Process vs Thread

Basis	Process	Thread
Memory	Separate memory space	Shared memory
Creation Cost	High	Low
Communication	IPC required	Direct communication
Isolation	Independent	Not independent
Example	Chrome & VS Code	Multiple tabs inside Chrome

Engineering Insight: Multithreading improves ML preprocessing speed.

4 Concurrency vs Multiprocessing

Basis	Concurrency	Multiprocessing
Meaning	Logical overlapping	Multiple CPUs working
Hardware Need	Not necessary	Required
Speed	Moderate	Faster
Example	Async programming	Parallel model training

5 Mutual Exclusion vs Deadlock

Basis	Mutual Exclusion	Deadlock
-------	------------------	----------

Meaning	Only one process allowed in critical section	Processes waiting forever
Purpose	Prevent race condition	Undesirable situation
Effect	Safe execution	System freeze
Example	One ATM at a time	Each process holding one fork

6 Deadlock vs Starvation

Basis	Deadlock	Starvation
Meaning	Circular waiting	Process waits indefinitely
Cause	Resource cycle	Low priority
Recovery	Needs external intervention	Priority adjustment
Example	Dining philosopher circular wait	Low priority process never scheduled

7 Binary Semaphore vs Counting Semaphore

Basis	Binary Semaphore	Counting Semaphore
Value Range	0 or 1	0 to N
Use	Mutual exclusion	Resource counting
Example	Lock	Parking lot slots

8 Shared Memory vs Message Passing

Basis	Shared Memory	Message Passing
Speed	Faster	Slower
Synchronization	Required	Less required
Complexity	Complex	Simple
Network Support	Same machine	Works across network
Example	Shared array	Pipe / Socket

9 Busy Waiting vs Blocking

Basis	Busy Waiting	Blocking
CPU Usage	Wastes CPU	CPU released
Efficiency	Low	High

Used In	Spinlocks	Semaphores
Example	While loop checking lock	Process sleeps

10 Dekker vs Peterson

Basis	Dekker	Peterson
Complexity	Complex	Simpler
Processes Supported	2	2
Mechanism	Flags + Turn	Flags + Turn
Practical Use	Rare	Theoretical learning